Computers & Operations Research 38 (2011) 1367–1376

Contents lists available at ScienceDirect

Computers & Operations Research

journal homepage: www.elsevier.com/locate/caor

Bee colony optimization for the *p*-center problem ☆

Tatjana Davidović [a], Dušan Ramljak [b], Milica Šelmić [c], Dušan Teodorović [c,*]

[a] Mathematical Institute, Serbian Academy of Sciences and Arts, Kneza Mihaila 36, P.O. Box 367, 11001 Belgrade, Serbia
[b] Center for Information Science and Technology, Temple University, 322 Wachman Hall, 1805 North Broad Street, Philadelphia, PA 19122, USA
[c] Faculty of Transport and Traffic Engineering, University of Belgrade, Vojvode Stepe 305, 11010 Belgrade, Serbia

ARTICLE INFO

Available online 17 December 2010

Keywords:
Meta-heuristics
Bee colony optimization
Combinatorial optimization
Location analysis
p-Center problem

ABSTRACT

Bee colony optimization (BCO) is a relatively new meta-heuristic designed to deal with hard combinatorial optimization problems. It is biologically inspired method that explores collective intelligence applied by the honey bees during nectar collecting process. In this paper we apply BCO to the *p*-center problem in the case of symmetric distance matrix. On the contrary to the constructive variant of the BCO algorithm used in recent literature, we propose variant of BCO based on the improvement concept (BCOi). The BCOi has not been significantly used in the relevant BCO literature so far. In this paper it is proved that BCOi can be a very useful concept for solving difficult combinatorial problems. The numerical experiments performed on well-known benchmark problems show that the BCOi is competitive with other methods and it can generate high-quality solutions within negligible CPU times.
© 2010 Elsevier Ltd. All rights reserved.

1. Introduction

Meta-heuristics have become very powerful tool for solving hard combinatorial optimization problems. Among them a class of biologically inspired algorithms can be recognized. Bee colony optimization (BCO) method, that explores collective intelligence applied by the honey bees during nectar collecting process, is one of them. BCO has been proposed by Lučić and Teodorović [1–3] and up to now it is successfully applied to various real-life optimization problems.

The BCO is a stochastic, random-search technique that belongs to the class of population-based algorithms. This technique uses an analogy between the way in which bees in nature search for food, and the way in which optimization algorithms search for an optimum of (given) combinatorial optimization problems.

Since its appearance it had various successful applications. BCO has been proposed for the first time in [1–3] for solving the travelling salesman problem and was evolving through later applications. The BCO has also been applied to: the vehicle routing problem [4], the routing and wavelength assignment (RWA) in all-optical networks [5], the ride-matching problem [6], the traffic sensors locations problem on highways [7], and the static scheduling of independent tasks on homogeneous multiprocessor systems [8].

In this paper we develop a new version of the BCO algorithm to deal with the problem of locating p centers on a network of n vertices (p-center problem) with symmetric distance matrix. *Center problems* in location analysis are the problems related to the location of emergency-type facilities (centers) in the transportation networks. Ambulance, fire and police departments all over the world have numerous facilities (fire stations, police stations, ambulances depots). Proper location of these facilities, as well as adequate vehicle dispatching strategy has a crucial influence on a survival rate of citizens who are in emergency-type situations. For emergency services, the location of the facilities must be such as to minimize the furthest distance (time) that the vehicles will ever travel when responding to an emergency call. The importance of this issue is reflected in the large number of papers that consider the location of centers [9–19].

To approach the p-center problem, we propose an artificial system composed of a number of precisely defined agents (individuals, artificial bees). Multi-agent simulation is performed, i.e. we specify behavior rules of our agents and simulate the interaction between them (the way in which agents communicate with each other). Our model and a corresponding computer program determine the way our agents perform their activities. Various behavior rules lead us to different variants of BCO algorithms. Two variants of the BCO algorithm can be distinguished straightforward: constructive BCO and BCOi, the variant based on the improvement concept. As of the authors' knowledge, the BCOi was not used in the relevant BCO literature so far, and here it proves to be very efficient. Therefore, the development of BCOi represents major contribution of this work.

To test the efficiency of our implementation we performed a set of numerical experiments on well-known benchmark problems. We compared BCOi results with the state-of-the-art heuristic

☆ This work has been supported by Serbian Ministry of Science and Technological Development, Grant nos. 144007 and 144033.
* Corresponding author. Tel.: +381 112890580; fax: +381 112638912.
E-mail addresses: tanjad@mi.sanu.ac.rs (T. Davidović), dusan.ramljak@temple.edu (D. Ramljak), m.selmic@sf.bg.ac.rs (M. Šelmić), duteodor@vt.edu, dusan@sf.bg.ac.rs (D. Teodorović).

0305-0548/$ - see front matter © 2010 Elsevier Ltd. All rights reserved.
doi:10.1016/j.cor.2010.12.002

algorithms. Reported results show that the BCOi is competitive with other methods with respect to both comparison criteria: solution quality and running time. Moreover, BCOi was able to generate three (out of 40) new best known solutions within negligible CPU times.

The rest of this paper is organized as follows. Section 2 is devoted to the description of BCO technique, while the description of p-center problem and actual implementation of BCOi for solving it are given in Section 3. Section 4 contains experimental evaluations and Section 5 concludes the paper.

2. Bee colony optimization (BCO)

The bee colony optimization (BCO) meta-heuristic [1–3] belongs to the class of *nature-inspired algorithms*—the algorithms based on various biological and natural processes. We are particularly interested in *swarm behavior* based on the biological needs of individuals to stay together.

When staying together, individuals have a higher chances to defend themselves, find food and therefore to stay alive. Typical examples of swarm behavior are flocks of birds, parliament of owls, herds of land animals, fish schools and various social insects (bees, wasps, ants, termites) colonies. Swarm behavior is primarily characterized by autonomy, distributed functioning and self-organizing. The communication systems between individual insects contribute to the pattern called the "collective intelligence" of the social insect colonies.

The term "swarm intelligence" that denotes this "collective intelligence", has been introduced in [20] and it is the branch of Artificial Intelligence. Swarm intelligence is based on investigation of actions of individuals in different decentralized systems. These decentralized systems (multi agent systems) are composed of physical individuals (robots, for example) or "virtual" (artificial) ones that communicate among themselves, cooperate, collaborate, exchange information and knowledge and perform some tasks in their environment. Scientists use principles discovered in nature to build artificial systems. The development of artificial systems usually does not involve the entire imitation of natural systems, but explores them while searching for ideas and models.

As the examples of artificial multi agent systems we mention ant colony optimization (ACO) [21], particle swarm optimization (PSO) [22] and various algorithms inspired by bees' behavior. BCO was among the first of bees-inspired algorithms, it appeared at the very beginning of this century. Among other algorithms inspired by honeybees we point out the following. Yonezava and Kikuchi [23] analyzed collective intelligence based on bees' behavior. Sato and Hagiwara [24] proposed modified genetic algorithm named bee system. In essence, this algorithm belongs to the class of genetic algorithms. Abbas [25] developed *MBO* model that is based on the marriage process in honeybees. *BeeHive* [26–28], *artificial bee colony* (ABC) algorithm [29–31] and *bees algorithm* [32–34] as well as BCO, are based on foraging behavior of honeybees, but all of them use different concepts for algorithm development. An interesting survey of the bees' behavior inspired algorithms could be found in [35,36].

The BCO meta-heuristic [1–3] has been proposed quite recently by Lučić and Teodorović (they used the term "bee system" in their first paper). The BCO is inspired by foraging behavior of honeybees. The basic plan behind the BCO is to build the multi agent system (colony of artificial bees) able to efficiently solve hard combinatorial optimization problems. The artificial bee colony, as any other nature-inspired object behaves partially similar, and partially in a different way from bee colonies in nature. The brief introduction to biological background of BCO follows in Section 2.1 and then BCO algorithm is described in details in Section 2.2.

2.1. BCO biological background

In order to describe basic idea of the BCO algorithm we are interested in principles and habits used by bees in nature when collecting food. Bee colonies in nature usually consist of 50,000–100,000 individual bees that cooperate in everyday activities. Majority of them are involved in exploring the fields in the neighborhood of their hive looking for food. They collect and accumulate nectar for later use.

Typically, in the initial step, some scouts search the region. Completing the search, scout bees return to the hive and inform their hive-mates about the locations, quantity and quality of available food sources in the areas they have examined. If they discover nectar in the previously investigated locations, scout bees dance in the so-called "dance floor area" of the hive, in an attempt to "advertise" food locations and encourage (recruit) the remaining members of the colony to follow their lead. The information about the food quantity is presented using a ritual called a "waggle dance". If a bee decides to leave the hive and collect nectar, it will follow one of the dancing scout bees to the previously discovered patch of flowers. Upon arrival, the foraging bee takes a load of nectar and returns to the hive relinquishing the nectar to a food storer.

Several scenarios are then possible for a foraging bee: (1) it can abandon the food location and take a role of an uncommitted follower; (2) it can continue with the foraging behavior at the discovered nectar source, without advertising it to the others; (3) it can try to recruit its (uncommitted) hive-mates with the dance ritual before the return to the food location. The bee opts for one of the above alternatives. As several bees may be attempting to recruit their hive-mates at the dance floor area at the same time, it is unclear how above decisions are made, although it has been concluded that "the recruitment among bees is always a function of the quality of the food source" [37]. The described process continues repeatedly, while the bees from a hive accumulate nectar and explore new areas with potential food sources.

2.2. BCO algorithm

The basic idea of designing BCO is to build the multi agent system (colony of artificial bees) that will search for good solutions of various combinatorial optimization problems exploring the principles used by honey bees during nectar collection process. Artificial bee colony usually consists of small number of individuals (for example 5 or 10), but nevertheless, BCO principles are gathered from natural systems. Artificial bees investigate through the search space looking for the feasible solutions. In order to find better and better solutions, autonomous artificial bees collaborate and exchange information. Using collective knowledge and sharing information among themselves, artificial bees concentrate on more promising areas, and slowly abandon solutions from the less promising ones. Step by step, artificial bees collectively generate and/or improve their solutions. The BCO search is running in iterations until some predefined stopping criteria is satisfied.

Population of agents (artificial bees) consisting of B bees collaboratively searches for the optimal solution. Every artificial bee is responsible for one solution to the problem. There are two alternating phases (*forward pass* and *backward pass*) constituting single step in the BCO algorithm. In each forward pass, every artificial bee explores the search space. It applies a predefined number of moves, which construct and/or improve the solution, yielding a new solution. For example, let bees Bee 1, Bee 2, . . ., Bee B participate in the decision-making process on n entities. At each forward pass bees are supposed to select one entity. Entity could be

one component of the partial solution according to the constructive version, or one complete solution that is going to be enhanced in the following algorithm phases, according to the BCOi. The possible situation after s-th forward pass is illustrated in Fig. 1. Rectangles in Fig. 1 represent partial/complete solutions associated to bees. Different patterns denote that each bee is associated with a different solution.

Having obtained new partial/complete solutions, the second phase, the so-called backward pass starts. In the backward pass, all artificial bees share information about the quality of their solutions. In nature, bees would return to the hive, perform a dancing ritual, which would inform other bees about the amount of food they have discovered, and the proximity of the patch to the hive. In the search algorithm, the bees announce the quality of the solution, i.e. the value of objective function is computed. When all solutions are evaluated, every bee decides with a certain probability whether it will stay loyal to its solution or not. The bees with better solutions have more chances to keep and advertise their solutions. Unlike the bees in nature, artificial bees that are *loyal* to their partial/complete solutions are at the same time *recruiters*, i.e. their solutions would be considered by other bees. Once its solution is abandoned, a bee becomes *uncommitted* and has to select one of the advertised solutions. This decision is taken with a probability in such a way that better among the advertised solutions have bigger opportunity to be chosen for further exploration. By this process, within each backward pass all bees are divided into two groups (R recruiters, and remaining $B-R$ uncommitted bees) as it is shown in Fig. 2. Values for R and $B-R$ are changing from one backward pass to another one.

Let, for example, after comparing all generated partial/complete solutions, Bee 1, from Fig. 1 decided to abandon its own solution and to join Bee B (see Fig. 3). Bee 1 and Bee B "fly together" along the path already generated by the Bee B. In practice, this means that partial/complete solution generated by Bee B is associated (copied) to Bee 1 also. Similarly, the Bee 4 was recruited by the Bee 2. After that, they are free to make their individual decisions about the next step to be made. In constructive BCO each bee adds a (different)

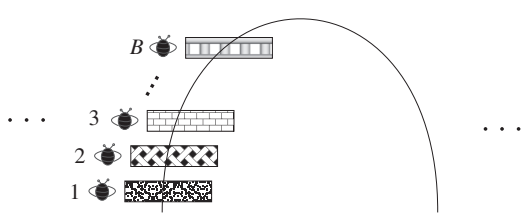

Fig. 1. An illustration of the s-th forward pass.

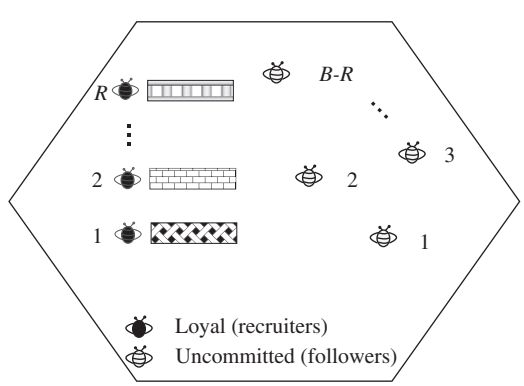

Fig. 2. Recruiting of uncommitted followers.

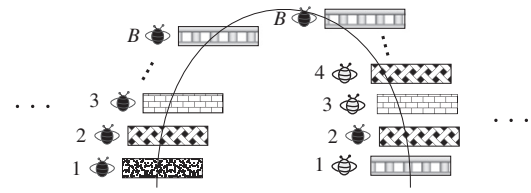

Fig. 3. The possible result of a recruiting process within s-th backward pass.

new component to the previously generated partial solution, while in the BCOi bees modify some components of the complete solution in order to enhance them. The Bee 3, from previous example, will keep already generated partial/complete solution without being chosen by any of the uncommitted hive-mates. The described situation is illustrated in Fig. 4.

The two phases of the search algorithm, forward and backward pass, are alternating NC times, i.e. until all required feasible solutions (one for each bee) are generated in constructive version, or until each bee performs NC solution modifications in BCOi. When all steps are completed the best solution is determined. It is then used to update global best solution and an iteration of the BCO is accomplished. At this point all B solutions are deleted, and the new iteration could start. The BCO runs iteration by iteration until a stopping condition is met. The possible stopping conditions could be, for example, the maximum total number of iterations, the maximum total number of iterations without the improvement of the objective function value, maximum allowed CPU time, etc. At the end, the best found solution (the so called global best) is reported as the final one.

The BCO algorithm parameters whose values need to be set prior the algorithm execution are as follows:

- B—The number of bees involved in the search.
- NC—The number of forward (backward) passes in a single iteration.

The following is the pseudo-code of the BCO algorithm:

Do

1. Initialization: a(n) (empty) solution is assigned to each bee.
2. *For* $(s=0; s<NC; s++)$ //count moves
   //**forward pass**
   (a) *For* $(b=0; b<B; b++)$
      (1) Evaluate all possible moves;
      (2) Choose one move using the roulette wheel.
   //**backward pass**
   (b) *For* $(b=0; b<B; b++)$
      Evaluate (partial/complete) solution for bee b;
   (c) *For* $(b=0; b<B; b++)$
      Loyalty decision using the roulette wheel for bee b;
   (d) *For* $(b=0; b<B; b++)$
      If (b is follower), choose a recruiter by the roulette wheel.
3. Evaluate all solutions and find the best one.

while stopping criteria is not satisfied.

The main difference between the BCO and other artificial multi agent systems is in the way agents communicate with each other: artificial bees communicate directly (sharing the information about quality of all solutions), while other agents use indirect communication (for example, ants use only the information that has been recorded by another agent already searched through the same path).

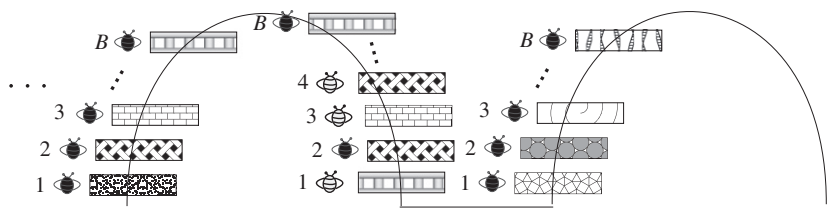

Fig. 4. An example of partial/complete solutions after (s+1)-th forward pass.

3. The BCO approach to *p*-center problem

In order to examine the performances of our BCOi algorithm we selected the p-center problem, one of the problems from the location analysis well known by its hardness. In this section, we describe our implementation of the BCOi to be applied to the p-center problem. First, we present the problem formulation along with the review of recent literature in Section 3.1. The BCOi implementation details are given in Section 3.2.

The constructive variant of BCO algorithm tailored for p-center problem was presented in [38]. In that variant bees build solutions through algorithm steps. Namely, in each forward pass the bees add *NC* new centers to the partial solutions, iteratively constructing the complete solutions.

In contrast to the constructive version, in BCOi bees are assigned complete solutions at the beginning, and modify them through the iterations. As of the authors' knowledge the BCOi concept has not been previously used in the relevant literature: all methods based on bees foraging principles are the constructive ones. Therefore, the contribution of this work to the evolution of the BCO algorithm is even greater.

It is important to note that within recent BCO implementations the artificial hive is a node without a precise location. Therefore, hive has no impact on the quality of solutions—it is only a fictitious place where bees exchange information.

3.1. The p-center problem formulation

The p-center problem is one of the fundamental problems in the area of discrete location. Due to its hardness and importance it has always been a challenge for scientists who approached it from different perspectives. In this paper we consider the case of symmetric distance matrix.

The p-center problem could be defined in the following way: Given is the set of n nodes (customers) and distances between any pair of nodes. This structure is usually referred to as *network*. The goal is to locate p facilities (centers) on a network in such a way to minimize the maximum of the distances from each node to its nearest center. We denote by p the total number of centers that should be located in the network. Centers could be located at any of the given n nodes. In order to describe mathematical formulation of the p-center problem we introduce the following notation:

- N—set of nodes (customers, locations) ($n=|N|$).
- $D=[d_{ij}]_{n\times n}$—Matrix containing distances between nodes obtained using *all shortest paths (Floyd's)* algorithm [39].

Let us introduce the binary variables defined in the following way:

$$x_{ij}=\begin{cases}1, & \text{if user from node } i \text{ is assigned to center located at node } j,\\ 0, & \text{otherwise.}\end{cases}$$

$$y_j=\begin{cases}1, & \text{if center is located at node } j,\\ 0, & \text{otherwise.}\end{cases}$$

The p-center problem could be formulated as integer linear program (ILP) in the following way [40]:

$$\min z \tag{1}$$

$$\text{s.t.}\quad z-\sum_{j=1}^{n} d_{ij}x_{ij}\geq 0,\quad 1\leq i\leq n, \tag{2}$$

$$\sum_{j=1}^{n} x_{ij}=1,\quad 1\leq i\leq n, \tag{3}$$

$$x_{ij}\leq y_j,\quad 1\leq i\leq n,\quad 1\leq j\leq n, \tag{4}$$

$$\sum_{j=1}^{n} y_j=p, \tag{5}$$

$$x_{ij},y_j\in\{0,1\},\quad 1\leq i\leq n,\ 1\leq j\leq n. \tag{6}$$

The objective function z that should be minimized represents the maximum over all the distances from each node to its nearest center as it is stated by constraints (2). Every demand node i should be assigned to exactly one potential center node j (constraints (3)). Constraints (4) assure that each demand node i cannot receive service at node j if that node is not already allocated as a center. The total number of centers is set to p by constraint (5). Constraints (6) describe binary nature of the problem variables.

The p-center problem is proven to be *NP*-hard [41]. For some special cases, exact algorithms have been proposed [12,40,42]. The p-center problem has also been solved using a new relaxation algorithms by Chen and Chen [19]. Relaxation algorithm is a method to optimally solve a large location problem by solving a succession of small sub-problems. It is an iterative algorithm which updates, at each step, bounds on the optimal solution, until the optimal solution is reached. Several heuristics and procedures based on meta-heuristic rules have been proposed by different authors [43,12,44,16,14,15,17]). Mladenović et al. [15] presented a basic variable neighborhood search and two tabu search heuristics for the p-center problem without triangle inequality. All proposed methods use the 1-interchange (namely vertex substitution) neighborhood structure. Pacheco and Casado [17] proposed a meta-heuristic procedure based on the scatter search approach for p-center problem and the maximum set covering problem with a small number of centers ($p\leq 10$).

We approached the p-center problem with the constructive variant of the BCO algorithm in [38]. Based on the results from [38] it can be concluded that constructive BCO algorithm manages to find the optimal solution only for small size examples (up to 50 nodes and 4 centers). When the dimension of the problem increases, constructive BCO fails comparing to the best known results from the literature. Deviation of BCO solutions from the best known results is sometimes greater than 10%. Such deviation occurs because the p-center problem contains a large dose of randomness and a large degree of combinatorial complexity (choice of p elements of n available) and heuristic concepts embedded in the construction of

solutions were not able to efficiently overcome these problems. Therefore, in this paper we propose the variant of the BCO based on the improvement concept—BCOi, which proved itself, as a very good tool to handle the p-center problem.

3.2. BCO based on improvement concept (BCOi)

As a consequence of unsatisfactory results obtained by applying constructive BCO, we decided to develop a new concept based on improving the *complete* solution held by each bee. We named this version of the BCO algorithm BCOi. As of the authors' knowledge this concept has not been explored in the literature so far, therefore we consider its development as the major contribution of our work. The BCOi implementation consists of the following five steps. The first step, called preprocessing is performed "off-line". The second step represents generation of initial complete solution. In the third, the most significant step, bees modify current solutions. Steps 4 (results comparison mechanism) and 5 (recruitment) are identical to the corresponding steps from the constructive BCO described in [38]. The last four steps are executed in the real CPU time.

3.2.1. Preprocessing

In this step the input data is transformed in order to reduce the time required for all computations performed online. All the activities could be divided into two phases:

1. In the first preprocessing phase the matrices D_{sort} and $D_{circles}$ are formed. We start from the matrix D and we sort every row of the matrix D in non-decreasing order along with corresponding indices. Therefore, each row of the matrix $D_{circles}$ informs us about the first, second, ...,n-th closest node to the observed node. For example, let $n=5$ and

$$D=\begin{bmatrix} 0 & 0.3 & 0.5 & 0.2 & 0.1 \\ 0.3 & 0 & 0.6 & 0.8 & 0.7 \\ 0.5 & 0.6 & 0 & 0.2 & 0.9 \\ 0.2 & 0.8 & 0.2 & 0 & 0.4 \\ 0.1 & 0.9 & 0.9 & 0.4 & 0 \end{bmatrix}.$$

The corresponding matrices D_{sort} and $D_{circles}$ that we get from the matrix D above are

$$D_{sort}=\begin{bmatrix} 0 & 0.1 & 0.2 & 0.3 & 0.5 \\ 0 & 0.3 & 0.6 & 0.7 & 0.8 \\ 0 & 0.2 & 0.5 & 0.6 & 0.9 \\ 0 & 0.2 & 0.2 & 0.4 & 0.8 \\ 0 & 0.1 & 0.4 & 0.9 & 0.9 \end{bmatrix}, \quad D_{circles}=\begin{bmatrix} 1 & 5 & 4 & 2 & 3 \\ 2 & 1 & 3 & 5 & 4 \\ 3 & 4 & 1 & 2 & 5 \\ 4 & 3 & 1 & 5 & 2 \\ 5 & 1 & 4 & 2 & 3 \end{bmatrix}.$$

2. In the second phase of preprocessing we generate matrix D_{radius}. All elements along main diagonal are equal to 1. All other elements (i,j) of the matrix D_{radius} represent the total number of nodes whose distance to node i is less then the distance between node i and node j. D_{radius} obtained from previous example is given as

$$D_{radius}=\begin{bmatrix} 1 & 3 & 4 & 2 & 1 \\ 1 & 1 & 2 & 4 & 3 \\ 2 & 3 & 1 & 1 & 4 \\ 1 & 4 & 1 & 1 & 3 \\ 1 & 3 & 3 & 2 & 1 \end{bmatrix}.$$

The roles of the above matrices are explained in the next subsection.

3.2.2. Initial solution generation

In accordance with the basic idea of BCOi, at the beginning of each iteration for all bees an initial (complete) solution has to be generated and evaluated. The role of this step is to introduce diversification into the search process.

We considered two possible variations for generating initial solutions. The first option was to generate an initial solution in a completely random manner. The second option (that yield better results) is based on the concept of *critical distance*. The critical distance represents the largest distance among all distances between nodes and their nearest centers. Centers were included in the initial solution one at a time. For each bee, the first center is selected randomly. When considering remaining nodes as candidates for a new center we first identify current critical distance (and corresponding node and center that define this distance).

We construct the circle with center located in the node that defines critical distance and radius equals critical distance (see Fig. 5). Obviously, the only way to improve the current partial solution is to select a node within that circle as the new center. We choose the new center among these nodes in a random manner. Matrices obtained through preprocessing steps help us to perform this selection easily. The element $D_{radius}(i,j)$ informs us about the total number of candidate nodes, t. These t nodes are identified as first t elements of the i-th row in the $D_{circles}$ matrix. We select randomly a number w between 1 and t and add node $D_{circles}(i,w)$ as a new center. More precisely, new center location is determined by the following formula:

$$j' = D_{circles}(i, rand(1, D_{radius}(i,j))), \tag{7}$$

where (i,j) denotes the pair (user, center) corresponding to the critical distance, and j' is the index of new center added to the current solution. Therefore, thanks to the preprocessing step, the complexity of adding new center is constant, i.e. $O(1)$.

For example, let us consider a network of 16 nodes (see Fig. 5), and let the total number of centers to be located equals 4. Let us assume that bee has already located centers in nodes 4, 7 and 13. The current objective function value is the distance from center 13 to user 16. We choose the new center in a random manner among nodes 14–16. If there are no nodes within the circumscribed circle, its center (i.e. user node corresponding to the critical distance) is adopted as "new" center. Described procedure is repeated until all centers are chosen.

When all the bees create one complete solution, the backward pass starts, consisting of the operations described in Sections 3.2.4 and 3.2.5.

3.2.3. Solution modification

The main step of the BCOi is modification of solution through NC forward passes within the single iteration. It proved to be the key factor that enables reaching the best possible solution quality. We designed this step in such a way to assure different treatment of same solutions.

Our modification consists of substituting some of the p centers with nodes selected from the remaining $n-p$ non-center nodes. We divide solution modification into two stages. The first stage consists of adding Q non centers (the solution feasibility is violated) in such a way to reduce critical distance. In the second stage, Q locations are removed from the center list in a greedy manner. Unlike local search based methods, BCOi does not perform systematic search for "better solutions". On the contrary, within each step (forward pass) BCOi selects for each bee one "neighboring solution" in randomly selected neighborhood Q. This neighbor is selected randomly among all neighbors with solution quality not worse than the current one.

The total number of new centers, Q, which will be added to the complete solution is determined in the random manner. By numerical experiments, we decided that Q should belong to the

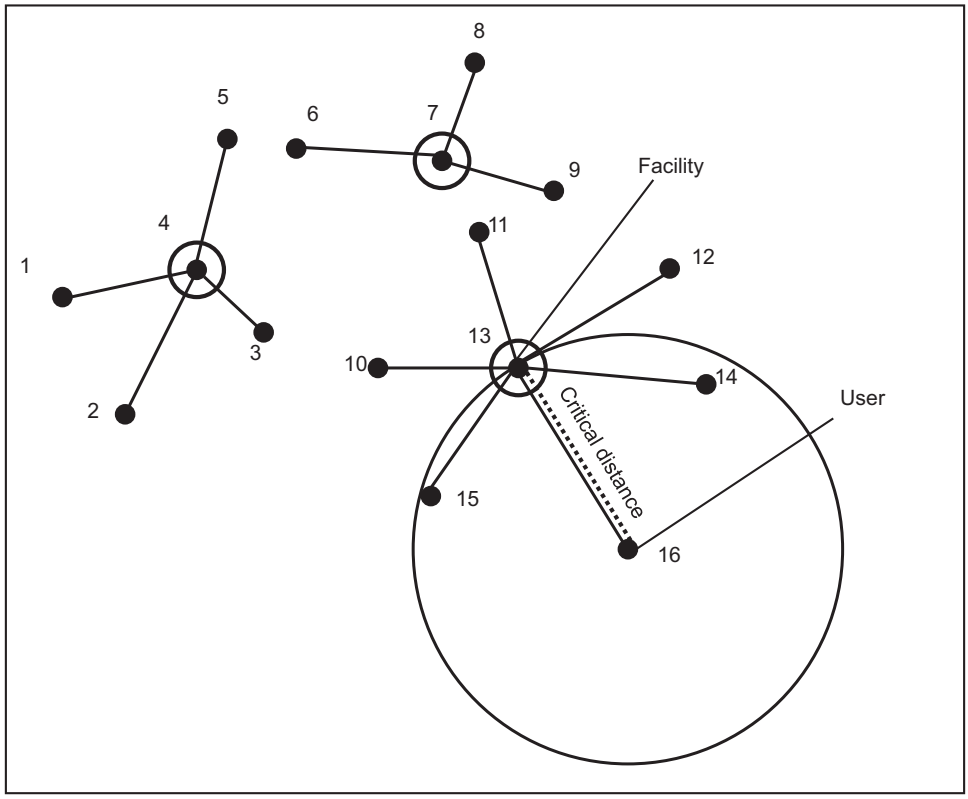

Fig. 5. Example of the "new" center selection.

following interval:

$$Q \in \begin{cases} [0,p], & \text{if } 5 \cdot p < n, \\ \left[0, \dfrac{n-2.5 \cdot p}{2.5}\right], & \text{otherwise.} \end{cases}$$

Once Q is determined for each bee, the selection of specific Q centers to be replaced can start. Procedure of reducing critical distance by adding new centers, described in Section 3.2.2, is repeated Q times. In this way, we create solution that contains $p+Q$ centers. Elimination of Q "surplus" centers from the solution is performed in the following way. For each of the $p+Q$ centers we examine the impact of the center to the solution quality. When the quality of the solution without that specific center is very bad, the observed center has high influence on the solution quality. If the solution quality does not deteriorate drastically by dropping the observed center, the center does not have a significant impact on the solution quality. The center with the smallest impact is removed from the center list. The described procedure is repeated Q times, until p centers remain. The greedy concept for removing centers is to assure that the final solution quality is not deteriorated with respect to the solution previously held by that particular bee.

Property. *The worst case complexity of solution modification procedure is* $O(np^2)$.

Proof. As it was described above, the solution transformation procedure consists of replacing Q centers with the same number of non-center locations. First, Q new non-center locations are added and it is performed by applying Eq. (7) Q times. Therefore, the complexity of this step is $O(1)$ and it does not influence the complexity of the whole procedure.

The second phase of the transformation procedure is removing Q locations following the greedy concept, i.e. we need to find locations that have the smallest impact to the solution quality. This means that we have to perform Q times the following steps: for each of current center locations ($O(p)$) we have to find new associated center ($O(p)$) to all users ($O(n)$) that were serviced at that center and evaluate the corresponding solution quality. Therefore, the worst case complexity of the second phase is $O(np^2)$. □

It is very important to note that in practice, the complexity of the solution modification procedure is reduced by the facts that not all users should be considered (just the ones associated to the center to be removed) and among the remaining centers one should test only the closest ones (according to the $D_{circles}$ matrix). Moreover, the two loops related to the centers are reducing during the execution process, i.e. the number of centers that are considered within the outer loop varies from $p+Q$ to $p+1$, while at the same time inner loop treats $p+Q-1$ centers at the beginning and p centers at the end.

After adding and dropping Q centers, forward pass is completed and each bee has the new complete solution. Backward pass, namely, the processes of solution evaluation, updating global best solution (if needed), loyalty testing and recruiting are performed as described in Sections 3.2.4 and 3.2.5. Different treatment of the same solutions held by recruiter and its follower(s) within the next forward pass is gained twofold. First because for each bee Q is selected randomly, reducing the probability that two bees will substitute the same number of centers. Even if Q is the same, random selection among t possible new centers within the circle defined by critical distance (see Eq. (7)), may assure that modifications are not the same.

The solution modification procedure as it is described in this subsection can be generalized and applied to any other combinatorial optimization problem. Main issues in this concept are: (1) to avoid time consuming local search procedure by defining efficient solution modification steps in such a way that the quality of resulting solution

is not deteriorated and (2) to introduce randomness with the aim to prevent same solutions being modified in the same way by different bees.

3.2.4. Results comparison mechanism

The evaluation and comparison of solutions associated to each bee is done after modifying solutions, at the beginning of each backward pass. The solutions are evaluated based on the maximum of all distances between the clients and the corresponding centers. When calculating these distances we assumed that every client receives service in the closest center.

Let us denote by z_b ($b=1, 2, \ldots, B$) the maximum distance between the clients and the centers in the case of the solution generated by the b-th bee. We call this maximum distance the *length*. We normalize the length z_b, and denote by O_b this normalized value, i.e.:

$$O_b = \frac{z_{max} - z_b}{z_{max} - z_{min}}, \quad O_b \in [0,1], \quad b = 1,2,\ldots,B, \tag{8}$$

where z_{min} and z_{max} are respectively minimum and maximum among all lengths produced by all bees.

The probability that the b-th bee (at the beginning of the new forward pass) is loyal to the previously discovered solution is calculated in this paper in the following way:

$$p_b^{u+1} = e^{-(O_{max} - O_b)/u}, \quad b = 1,2,\ldots,B, \tag{9}$$

where u is the counter (ordinary number) of the forward pass (taking values 1,2,…,NC) and O_{max} is the maximum among all O_b values (the normalized value of the best objective function value among all bees' solutions). Let us discuss relation (9) in some more details. The better the generated solution (higher O_b value), the higher the probability that the bee will be loyal to the previously discovered solution. The greater the ordinary number of the forward pass, the higher the influence of the already discovered solution. This is expressed by the term u in the denominator of the exponent (relation (9)). In other words, at the beginning of the search process bees are "more brave" to search the solution space. The more forward passes they make, the bees have less courage to explore the solution space. The more we are approaching the end of the search process, the bees are more focused on the already known solutions. Using relation (9) and a random number generator, every artificial bee decides to become uncommitted follower, or to continue exploring its own solution.

3.2.5. Recruitment

If, at the beginning of $(u+1)st$ forward pass, bee does not want to explore its previous solution, it then has to select and follow the solution from another bee. In this paper, we assume the probability that the solution of a recruiter b will be chosen by any uncommitted bee is equal:

$$p_b = \frac{O_b}{\sum_{k=1}^{R} O_k}, \quad b = 1,2,\ldots,R, \tag{10}$$

where O_k—objective function value of the k-th advertised solution; R—the number of recruiters.

Using relation (10) and a random number generator, every uncommitted follower joins one recruiter. The recruiter's solution is copied to a solution of uncommitted follower. At this point all bees are free to again independently search the solution space. Recruiter's solution becomes follower's one, and after that bees apply different modification steps to the same solution.

An iteration completes when forward and backward passes are performed NC times. Best among all solutions is identified and used to update the global best one. All solutions are deleted (data structure cleaned) and the new iteration can start. The BCOi runs iteration by iteration until a stopping condition is met. In this paper we use maximum allowed CPU time as a stopping criterion. The results we obtained using the above described algorithm are presented in the next section.

4. Experimental evaluation

The BCOi is compared with state of the art techniques, i.e. the algorithms used for p-center problem that proved to be the best so far in the literature: multistart interchange (M-I), variable neighborhood search (VNS), tabu search 1 (TS-1) and tabu search 2 (TS-2) proposed in [15] and scatter search (SS) proposed in [17]. The comparison criteria are solution quality and running time.

4.1. Computing environment

Comparison is performed on the 40 test examples, originally designed for testing the p-median problem (OR-Lib Test Problems [13]). The problem parameters: n ranges from 100 to 900; p takes values between 5 and 200 ($p \le n/3$).

For all examples, initial solutions are generated based on the "critical distance concept", as it is explained in Section 3.2.2. Within BCOi algorithm we set $B=5$, $NC=10$. The values are selected based on our previous experience. These values turn out to be both relatively small (resulting in quick completion of each iteration) and large enough to assure efficient knowledge exchange and generation of high quality final solutions. We performed verification of our selection for BCOi parameters as it is presented in Section 4.3.

The stopping criterion is the maximum allowed CPU time. This time is different for each problem, and varies from 1 up to 20 s. It is determined in such a way as to assure fair comparison of different algorithms running on different machines. Namely, our heuristic implementation is running on Intel Core 2 Duo CPU E6750 on 2.66 GHz with $RAM=8$ Gb under Linux Slackware 12, Kernel: 2.6.21.5, gcc version 4.1.2. BCOi approach is implemented in C programming language with g++ compiler. The algorithms from [15] were tested on SUN Sparc Station 10, while the authors of [17] used Pentium 3, 600 MHz. When comparing the running times obtained by different machines it is necessary to perform the proper scaling of the results. The benchmark examples from the site `www.spec.org` are used for measuring and comparing compute-intensive floating point performances. Considering this, we observe that all techniques' running times from [15] should be divided by 120. Results from [17] were first transformed to be on the same scale as [15] and then divided by 120. This transformation needed to be performed for each instance considered in [17]. For example if the best SS time in [17] is reported to be 0.06 for *pmed*6, the corresponding time for TS-2 in [17] is 0.12 then their ratio equals 2. Therefore, we divide the reported time for TS-2 in [15] by 2 (to estimate SS time) and then by 120 to get the best running time results for the *pmed*6.

4.2. Results

The results of comparison between our BCOi and the best performing method from the literature are summarized in Table 1. We used M-I, VNS, TS-1 and TS-2 presented in [15] and SS presented in [17]. Since for each example at least one of the algorithms M-I, VNS, TS-1, TS-2, SS, generated best known solution from the literature, we always compare BCOi solution quality with that best known value. When comparing running times, BCOi is compared with the fastest algorithm from [15,17].

The first column of the Table 1 contains the name of the *pmed* example. The total number of nodes and the total number of centers are given in the next two columns. The fourth column contains the

Table 1
The BCOi results on all *pmed* examples with five bees and $NC=10$.

Example	n	p	Best known	Best BCOi	Best t (s) in lit.	Best t (s) by BCOi
pmed1	100	5	127.00	127.00	**0.000000**	**0.000000**
pmed2	100	10	98.00	98.00	0.000284	**0.000000**
pmed3	100	10	93.00	93.00	0.000080	**0.000000**
pmed4	100	20	74.00	74.00	0.002500	**0.000000**
pmed5	100	33	48.00	48.00	0.000250	**0.000000**
pmed6	200	5	84.00	84.00	0.000083	**0.000000**
pmed7	200	10	64.00	64.00	0.000356	**0.000000**
pmed8	200	20	55.00	55.00	**0.008750**	0.010000
pmed9	200	40	37.00	37.00	0.018333	**0.000000**
pmed10	200	67	20.00	20.00	0.075333	**0.030000**
pmed11	200	5	59.00	59.00	0.012667	**0.000000**
pmed12	300	10	51.00	51.00	**0.011500**	0.020000
pmed13	300	30	36.00	37.00	0.011250	**0.010000**
pmed14	300	60	26.00	27.00	0.323333	**0.140000**
pmed15	300	100	18.00	18.00	0.052500	**0.040000**
pmed16	400	5	47.00	47.00	**0.000000**	**0.000000**
pmed17	400	10	39.00	39.00	**0.002083**	0.010000
pmed18	400	20	28.00	29.00	0.161083	**0.150000**
pmed19	400	80	19.00	19.00	1.008833	**0.070000**
pmed20	400	133	14.00	14.00	0.435417	**0.090000**
pmed21	500	5	40.00	40.00	0.000250	**0.000000**
pmed22	500	10	38.00	39.00	0.297917	**0.190000**
pmed23	500	50	23.00	23.00	**0.069167**	0.230000
pmed24	500	100	16.00	16.00	0.229500	**0.080000**
pmed25	500	167	12.00	12.00	1.463917	**0.380000**
pmed26	600	5	38.00	38.00	0.000750	**0.000000**
pmed27	600	10	32.00	32.00	**0.002500**	0.010000
pmed28	600	60	19.00	19.00	0.209083	**0.050000**
pmed29	600	120	13.00	14.00	1.260667	**1.140000**
pmed30	600	200	11.00	**10.00**	0.705417	**0.350000**
pmed31	700	5	30.00	30.00	0.001083	**0.000000**
pmed32	700	10	29.00	29.00	0.225583	**0.060000**
pmed33	700	70	16.00	16.00	0.800667	**0.720000**
pmed34	700	140	12.00	12.00	0.673417	**0.390000**
pmed35	800	5	30.00	30.00	0.015417	**0.010000**
pmed36	800	10	27.00	28.00	0.421667	**0.080000**
pmed37	800	80	16.00	16.00	0.825000	**0.120000**
pmed38	900	5	29.00	29.00	0.003750	**0.000000**
pmed39	900	10	23.00	24.00	0.042113	**0.010000**
pmed40	900	90	14.00	14.00	0.397833	**0.190000**

best known result from the literature. Next column presents the best results reached by the BCOi algorithm. In the sixth column we report only the shortest running time used to obtain the best known result by any of M-I, VNS, TS-1, TS-2 or SS. Seventh column contains the running times required to obtain the best BCOi result from a single run. All reported running times are in seconds.

The performed numerical experiments show that the BCOi was able to improve best known solution in one of the tested examples (*pmed*30). It also obtained best known solutions from the literature in 32 examples (out of 40). For almost all these examples (29 out of 33), BCOi running times were smaller then the ones required by the fastest algorithm from the literature. From Table 1 it can also be seen that for the remaining seven examples BCOi obtained second best value for the objective function within negligible running time.

In order to examine the stability of our BCOi, we repeated execution 1000 times for each test example. The obtained results are summarized in Table 2. The first column of Table 2 contains the name of *pmed* example. In the second column the best known result is given. The number of reached best known results is in the third column. Next two columns show the average objective function values obtained through all 1000 executions and their standard deviations. The standard deviation is calculated in the following way:

$$\sigma = \sqrt{\frac{1}{No}\sum_{l=1}^{No}(z_l-\mu)^2},$$

where No is number of executions, z_l is the objective function value attained in the l-th execution and μ is the average objective function value. Column six contains average running times needed by BCOi to reach the final solution in each of 1000 executions. Allowed CPU time (stopping criteria) in each repetition is given in the last column.

The most important conclusion that comes from the results presented in Table 2 is that BCOi was able to obtain new best known solutions for three *pmed* examples. They are presented in bold in the second column. It is very interesting that in all of these examples number of centers is large. Moreover, from Table 2 it can be seen that BCOi meta-heuristic exhibits stable performances. In the large number of benchmark examples BCOi generated the best known solutions in almost all executions. In benchmark problems in which BCOi reached only a few best known results, in almost all other executions the obtained objective function value was the second best.

4.3. Parametric analysis

To verify our selection of the BCOi parameter values we selected the example *pmed*9 (the one for which best known solution is obtained in almost 50% of the executions) and performed 1000 repetitions of BCOi for different combinations of parameters. For this purpose we let B to take the following values: 1, 5, 10, 20, 30, and at the same time we varied NC from 10 to 30 with the

Table 2
The BCOi stability evaluation—1000 repetitions for *pmed* examples.

Example	Best known	No. BK	μ	σ	Aver. t (s)	CPU t (s)
pmed1	127.00	1000	127.00	0.00	0.00	1.00
pmed2	98.00	996	98.00	0.08	0.37	2.00
pmed3	93.00	285	93.72	0.45	0.38	2.00
pmed4	74.00	988	74.02	0.16	0.44	2.00
pmed5	48.00	1000	48.00	0.00	0.00	1.00
pmed6	84.00	996	84.00	0.06	0.35	2.00
pmed7	64.00	1000	64.00	0.00	0.15	2.00
pmed8	55.00	328	55.84	0.68	0.72	2.00
pmed9	37.00	455	37.57	0.54	0.69	2.00
pmed10	20.00	1000	20.00	0.00	0.29	2.00
pmed11	59.00	993	59.01	0.08	0.39	2.00
pmed12	51.00	390	51.61	0.49	0.41	2.00
pmed13	36.00	1	37.12	0.32	0.62	2.00
pmed14	26.00	9	27.18	0.41	1.08	3.00
pmed15	18.00	1000	18.00	0.00	0.14	6.00
pmed16	47.00	1000	47.00	0.00	0.02	1.00
pmed17	39.00	948	39.05	0.22	0.62	2.00
pmed18	28.00	1	29.09	0.28	15.02	70.00
pmed19	19.00	414	19.59	0.49	1.09	4.00
pmed20	14.00	1000	14.00	0.00	0.56	4.00
pmed21	40.00	1000	40.00	0.00	0.14	2.00
pmed22	38.00	1	39.01	0.11	0.45	2.00
pmed23	23.00	239	23.76	0.43	0.77	4.00
pmed24	16.00	679	16.32	0.47	1.65	5.00
pmed25	**11.00**	32	11.97	0.18	0.80	4.00
pmed26	38.00	993	38.01	0.08	0.37	2.00
pmed27	32.00	887	32.11	0.32	0.63	2.00
pmed28	19.00	998	19.00	0.04	0.93	4.00
pmed29	13.00	3	14.00	0.05	0.49	10.00
pmed30	**10.00**	1000	10.00	0.00	1.17	8.00
pmed31	30.00	1000	30.00	0.00	0.15	2.00
pmed32	29.00	49	29.95	0.22	0.16	2.00
pmed33	16.00	237	16.76	0.43	0.98	10.00
pmed34	**11.00**	45	11.96	0.21	0.62	15.00
pmed35	30.00	795	30.20	0.40	0.57	2.00
pmed36	27.00	3	28.00	0.07	0.31	2.00
pmed37	16.00	947	16.05	0.22	1.68	7.00
pmed38	29.00	1000	29.00	0.00	0.05	2.00
pmed39	23.00	2	24.00	0.04	0.27	20.00
pmed40	14.00	997	14.00	0.05	1.96	20.00

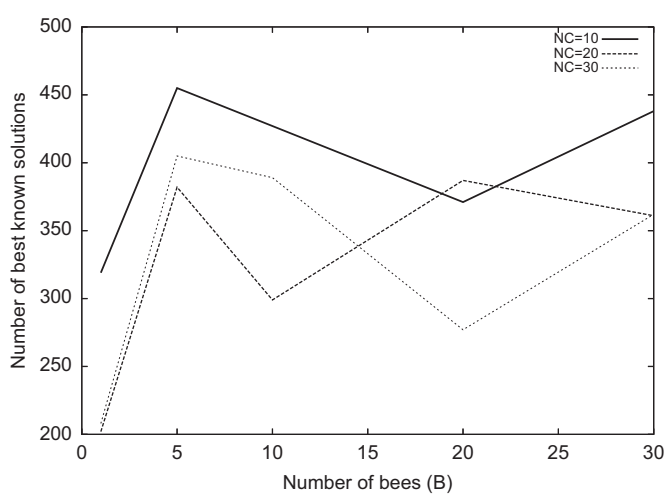

Fig. 6. The influence of BCOi parameters to the solution quality.

increment 10. For each execution we recorded the number of best known solutions (out of the 1000 obtained ones) and used them for drawing graphs given in Fig. 6.

As it can be seen from Fig. 6, the selected combination of parameters ($B=5$, $NC=10$) seems to be the most efficient one: it yields the largest number of obtained best known solutions (455 out of 1000 repetitions). Setting $B=1$, i.e. working with a single solution at a time, showed very low performance (see starting point of each curve in Fig. 6). This is easy to explain by the fact that in this case BCOi is performing search procedure which is something like *multistart local search*, i.e. collective intelligence, the basic information exchange step of BCOi, is not performed at all. From Fig. 6 it is also obvious that increasing either B or NC does not improve efficiency of our BCOi implementation.

5. Conclusion

The BCO is meta-heuristic technique created by the analogy with foraging behavior of honeybees, realizing the concepts of collective intelligence. A population of artificial bees searches for the optimal solution with cooperation that enables more efficiency and allows bees to reach the goals they could not achieve individually.

In this paper the BCO heuristic algorithm is used to tackle the p-center problem. Since the previously exploited constructive concept did not result in satisfactory solution qualities for benchmark examples, we developed BCOi—the improvement concept of BCO. As of the authors' knowledge our BCOi implementation is the first one exploring solution modification process instead of constructing new solutions from scratch. Therefore, the development of BCOi represent the main contribution of this work.

The proposed BCOi algorithm is tested on various benchmark examples. Based on the obtained results, we can conclude that BCOi is able to produce high quality solutions within negligible CPU times. We compared BCOi performances with other techniques from relevant literature and proved that BCOi is very competitive. Comparing to state of the art techniques, BCOi generated new best known solutions for three examples (out of 40). Moreover, running times of BCOi are significantly smaller than the corresponding times required by the state-of-the-art methods. The achieved results indicate that the development of new models based on swarm intelligence principles could considerably contribute to the solution of difficult location analysis problems.

There are no theoretical results at this moment that could support proposed approach. Good experimental results obtained by performed numerical evaluations represent strong motivation to try to produce some theoretical background. Beside the implementation of the BCO algorithm for other hard combinatorial problems, one of the most important aspects of future research is the mathematical justification of the BCO and BCOi technique.

Acknowledgment

The authors would like to thank the anonymous referees for the valuable suggestions that helped to significantly improve the presentation of the results described in this paper.

References

[1] Lučić P, Teodorović D. Bee system: modeling combinatorial optimization transportation engineering problems by swarm intelligence. In: Preprints of the TRISTAN IV triennial symposium on transportation analysis, Sao Miguel, Azores Islands; 2001. p. 441–5.
[2] Lučić P, Teodorović D. Transportation modeling: an artificial life approach. In: Proceedings of the 14th IEEE international conference on tools with artificial intelligence, Washington, DC; 2002. p. 216–23.
[3] Lučić P, Teodorović D. Computing with bees: attacking complex transportation engineering problems. International Journal on Artificial Intelligence Tools 2003;12:375–94.
[4] Lučić P, Teodorović D. Vehicle routing problem with uncertain demand at nodes: the bee system and fuzzy logic approach. In: Verdegay JL, editor. Fuzzy Sets Based Heuristics for Optimization. Berlin, Heidelberg: Physica Verlag; 2003. p. 67–82.

[5] Marković G, Teodorović D, Aćimović-Raspopović V. Routing and wavelength assignment in all-optical networks based on the bee colony optimization. AI Communications 2007;20:273–85.
[6] Teodorović D, Dell'Orco M. Mitigating traffic congestion: solving the ride-matching problem by bee colony optimization. Transportation Planning and Technology 2008;31:135–52.
[7] Edara P, Šelmić M, Teodorović D. Heuristic solution algorithms for a traffic sensor optimization problem. In: INFORMS 2008, Washington, DC; 2008.
[8] Davidović T, Šelmić M, Teodorović D. Scheduling independent tasks: bee colony optimization approach. In: Proceedings of the 17th Mediterranean conference on control and automation, Makedonia Palace, Thessaloniki, Greece; 2009. p. 1020–5.
[9] Hakimi SL. Optimal locations of switching centers and the absolute centers and medians of a graph. Operations Research 1964;12:450–9.
[10] Hakimi SL. Optimum distribution of switching centers in a communication network and some related graph theoretic problems. Operations Research 1965;13(3):462–75.
[11] Minieka E. The centers and medians of a graph. Operations Research 1977;25(4):641–50.
[12] Drezner Z. The planar two center and two median problems. Transportation Science 1984;18:451–61.
[13] Beasly JE. A note on solving large *p*-median problems. European Journal of Operational Research 1985;21:270–3.
[14] Hassin R, Levin M, Morad D. Lexicographic local search and the p-center problem. European Journal of Operational Research 2003;151:265–79.
[15] Mladenović N, Labbe M, Hansen P. Solving the p-center problem with tabu search and variable neighborhood search. Networks 2003;42(1):48–64.
[16] Caruso C, Colorni A, Aloi L. Dominant, an algorithm for the p-center problem. European Journal of Operational Research 2003;149(1):53–64.
[17] Pacheco JA, Casado S. Solving two location models with few facilities by using a hybrid heuristic: a real health resources case. Computers & Operations Research 2005;32(12):3075–91.
[18] ReVelle C, Eiselt H. Location analysis: a synthesis and survey. European Journal of Operational Research 2005;165(1):1–19.
[19] Chen D, Chen R. New relaxation-based algorithms for the optimal solution of the continuous and discrete p-center problems. Computers & Operations Research 2009;36(5):1646–55.
[20] Bonabeau E, Dorigo M, Theraulaz G. Swarm intelligence. Oxford: Oxford University Press; 1997.
[21] Dorigo M, Stützle T. Ant colony optimization. The MIT Press; 2004.
[22] Kennedy J, Eberhart R. Particle swarm optimization. In: Proceedings of IEEE international conference on neural networks IV; 1995. p. 1942–8.
[23] Yonezawa Y, Kikuchi T. Ecological algorithm for optimal ordering used by collective honey bee behavior. In: Proceedings of the seventh international symposium on micro machine and humane science, Nagoya, Japan; 1995. p. 249–55.
[24] Sato T, Hagiwara M. Bee system: finding solution by a concentrated search. In: Proceedings of the IEEE international conference on systems, man, and cybernetics 'computational cybernetics and simulation', Orlando, FL, USA; 1997. p. 3954–9.
[25] Abbass HA. MBO: marriage in honey bees optimization-a haplometrosis polygynous swarming approach. In: Proceedings of the congress on evolutionary computation, Seoul, South Korea; 2001. p. 207–14.
[26] Wedde HF, Farooq M, Zhang Y. BeeHive: an efficient fault-tolerant routing algorithm inspired by honey bee behavior. In: Ant colony optimization and swarm intelligence. Lecture notes in computer science, vol. 3172; 2004. p. 83–94.
[27] Wedde HF, Timm C, Farooq M. Beehiveais: a simple, efficient, scalable and secure routing framework inspired by artificial immune systems. In: Lecture notes in computer science, vol. 4193; 2006. p. 623–32.
[28] Wedde HF, Sebastian L, Bernhard van B, Zeynep B. A novel class of multi-agent algorithms for highly dynamic transport planning inspired by honey bee behavior. In: Proceedings of the 12th IEEE international conference on factory automation, Patras, Greece; 2007. p. 1157–64.
[29] Karaboga D. An idea based on honey bee swarm for numerical optimization. Technical Report, Erciyes University, Engineering Faculty Computer Engineering Department Kayseri/Turkiye; 2005.
[30] Karaboga D, Basturk B. A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. Journal of Global Optimization 2007;39:459–71.
[31] Karaboga D, Basturk B. On the performance of artificial bee colony algorithm. Applied Soft Computing 2008;8:687–97.
[32] Pham DT, Ghanbarzadeh A, Koc E, Otri S, Zaidi M. The bees algorithm—a novel tool for complex optimisation problems. In: Proceedings of the 2nd virtual international conference on intelligent production machines and systems (IPROMS 2006). Cardiff: Elsevier; 2006. p. 454–9.
[33] Pham DT, Soroka AJ, Ghanbarzadeh A, Koc E. Optimising neural networks for identification of wood defects using the bees algorithm. In: Proceedings of the IEEE international conference on industrial informatics, Singapore; 2006, pp. 1346–51.
[34] Pham DT, Haj Darwish A, Eldukhr EE. Optimisation of a fuzzy logic controller using the bees algorithm. International Journal of Computer-aided Engineering Technology 2009:250–64.
[35] Karaboga D, Akay B. A survey: algorithms simulating bee swarm intelligence. Artificial Intelligence Review 2009;31:61–85.
[36] Teodorović D. Bee colony optimization (BCO). In: Lim CP, Jain LC, Dehuri S, editors. Innovations in swarm intelligence. Berlin, Heidelberg: Springer-Verlag; 2009. p. 39–60.
[37] Camazine S, Sneyd J. A model of collective nectar source by honey bees: self-organization through simple rules. Journal of Theoretical Biology 1991;149:547–71.
[38] Teodorović D, Davidović T, Šelmić M, Ramljak D. An application of a meta-heuristic algorithm to p-center location problem. In: Proceedings of the symposium on information technology, YUINFO, Kopaonik, Serbia (in Serbian on CD 026.pdf); 2010.
[39] Lawler E. Combinatorial optimization: networks and matroids. Holt, Rinehart and Winston; 1976.
[40] Daskin M. Network and discrete location: models, algorithms, and application. John Wiley & Sons; 1995.
[41] Kariv O, Hakimi SL. An algorithmic approach to network location problems. Part ii. The *p*-median. SIAM Journal of Applied Mathematics 1969;37:539–60.
[42] Handler GY. *p*-center problems. In: Mirchandani P, Francis RL, editors. Discrete location theory. John Wiley & Sons; 1990. p. 305–47.
[43] Chandrasekaran R, Tamir A. Polynomial bounded algorithms for locating *p*-centers on a tree. Mathematical Programming 1982;22:304–15.
[44] Pelgrin B. Heuristic methods for the p-center problem. RAIRO Recherche Operationelle 1991;25:65–72.